

OOP_2

Sugeneruota Doxygen 1.9.8

Chapter 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Zmogus	??
Studentas	??

Chapter 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	??
Zmogus	??

Chapter 3

Failo Indeksas

3.1 Failai

Visų failų sąrašas su trumpais aprašymais:

include/ my_lib.h	??
include/ Studentas.h	??
include/ Zmogus.h	??
src/ main.cpp	??
src/ my_lib.cpp	??

Chapter 4

Klasės Dokumentacija

4.1 Studentas Klasė

#include <Studentas.h>

Paveldimumo diagrama Studentas:

Bendradarbiavimo diagrama Studentas:

Vieši Metodai

- `Studentas ()`
- `Studentas (const std::string &vardas, const std::string &pavarde, int egz, const std::vector< int > &nd)`
- `Studentas (const Studentas &s)`
- `Studentas (Studentas &&s)`
- `~Studentas ()` override
- `Studentas & operator= (const Studentas &s)`
- `Studentas & operator= (Studentas &&s)`
- `std::string tipas ()` const override
- `int getEgz ()` const
- `const std::vector< int > & getNd ()` const
- `double getGalVid ()` const
- `double getGalMed ()` const
- `void setEgz (int e)`
- `void setGalVid (double v)`
- `void setGalMed (double v)`
- `void pridetiNd (int pazymys)`
- `void isvalytiNd ()`
- `void pasalintiPaskutiniNd ()`
- `double skaiciuotiGalutiniSuVid ()` const
- `double skaiciuotiGalutiniSuMed ()` const

Vieši Metodai inherited from `Zmogus`

- `Zmogus ()`
- `Zmogus (std::string vardas, std::string pavarde)`
- `virtual ~Zmogus ()`
- `const std::string & getVardas ()` const
- `const std::string & getPavarde ()` const
- `void setVardas (const std::string &v)`
- `void setPavarde (const std::string &p)`

Draugai

- `std::ostream & operator<< (std::ostream &out, const Studentas &s)`
- `std::istream & operator>> (std::istream &in, Studentas &s)`

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- `std::string vardas_`
- `std::string pavarde_`

4.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.1.1.1 Studentas() [1/4]

```
Studentas::Studentas ( ) [inline]
```

4.1.1.2 Studentas() [2/4]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    int egz,
    const std::vector< int > & nd ) [inline]
```

4.1.1.3 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & s ) [inline]
```

4.1.1.4 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && s ) [inline]
```

4.1.1.5 ~Studentas()

```
Studentas::~~Studentas ( ) [inline], [override]
```

4.1.2 Metodų Dokumentacija

4.1.2.1 getEgz()

```
int Studentas::getEgz ( ) const [inline]
```

4.1.2.2 getGalMed()

```
double Studentas::getGalMed ( ) const [inline]
```

4.1.2.3 getGalVid()

```
double Studentas::getGalVid ( ) const [inline]
```

4.1.2.4 getNd()

```
const std::vector< int > & Studentas::getNd ( ) const [inline]
```

4.1.2.5 isvalytiNd()

```
void Studentas::isvalytiNd ( ) [inline]
```

4.1.2.6 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & s ) [inline]
```

4.1.2.7 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && s ) [inline]
```

4.1.2.8 pasalintiPaskutiniNd()

```
void Studentas::pasalintiPaskutiniNd ( ) [inline]
```

4.1.2.9 pridetiNd()

```
void Studentas::pridetiNd (
    int pazymys ) [inline]
```

4.1.2.10 setEgz()

```
void Studentas::setEgz (
    int e ) [inline]
```

4.1.2.11 setGalMed()

```
void Studentas::setGalMed (
    double v ) [inline]
```

4.1.2.12 setGalVid()

```
void Studentas::setGalVid (
    double v ) [inline]
```

4.1.2.13 skaiciuotiGalutiniSuMed()

```
double Studentas::skaiciuotiGalutiniSuMed ( ) const
```

4.1.2.14 skaiciuotiGalutiniSuVid()

```
double Studentas::skaiciuotiGalutiniSuVid ( ) const
```

4.1.2.15 tipas()

```
std::string Studentas::tipas ( ) const [inline], [override], [virtual]
```

Realizuoja [Zmogus](#).

4.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.1.3.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s ) [friend]
```

4.1.3.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas & s ) [friend]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [include/Studentas.h](#)
- [src/my_lib.cpp](#)

4.2 Zmogus Klasė

```
#include <Zmogus.h>
```

Paveldimumo diagrama Zmogus:

Vieši Metodai

- `Zmogus ()`
- `Zmogus (std::string vardas, std::string pavarde)`
- `virtual ~Zmogus ()`
- `virtual std::string tipas () const =0`
- `const std::string & getVardas () const`
- `const std::string & getPavarde () const`
- `void setVardas (const std::string &v)`
- `void setPavarde (const std::string &p)`

Apsaugoti Atributai

- `std::string vardas_`
- `std::string pavarde_`

4.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.2.1.1 Zmogus() [1/2]

```
Zmogus::Zmogus ( ) [inline]
```

4.2.1.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    std::string vardas,
    std::string pavarde ) [inline]
```

4.2.1.3 ~Zmogus()

```
virtual Zmogus::~Zmogus ( ) [inline], [virtual]
```

4.2.2 Metodų Dokumentacija

4.2.2.1 getPavarde()

```
const std::string & Zmogus::getPavarde ( ) const [inline]
```

4.2.2.2 getVardas()

```
const std::string & Zmogus::getVardas ( ) const [inline]
```

4.2.2.3 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p ) [inline]
```

4.2.2.4 setVardas()

```
void Zmogus::setVardas (
    const std::string & v ) [inline]
```

4.2.2.5 tipas()

```
virtual std::string Zmogus::tipas ( ) const [pure virtual]
```

Realizuota [Studentas](#).

4.2.3 Atributų Dokumentacija

4.2.3.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

4.2.3.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- include/[Zmogus.h](#)

Chapter 5

Failo Dokumentacija

5.1 include/my_lib.h Failo Nuoroda

```
#include "Studentas.h"
#include <string>
#include <vector>
#include <deque>
#include <list>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <stdexcept>
#include <iostream>
#include <iomanip>
#include <chrono>
#include <type_traits>
```

Įtraukimo priklausomybių diagrama my_lib.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Funkcijos

- void [rodytiRezultatus](#) (const std::vector< [Studentas](#) > &studentai)
- void [generuotiPazymius](#) ([Studentas](#) &s)
- void [generuotiStudentus](#) (std::vector< [Studentas](#) > &studentai)
- void [rodytiRez](#) (const std::vector< [Studentas](#) > &studentai)
- bool [rikiuotiVarda](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- bool [rikiuotiPavarde](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- bool [rikiuotiPagalGalutiniVid](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- bool [rikiuotiPagalGalutiniMed](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- void [nuskaitytiFailaTestavimui](#) (std::vector< [Studentas](#) > &studentai, int kartai)
- void [generuotiFaila](#) (int studentuSk)
- void [investiDuFailus](#) (const std::vector< [Studentas](#) > &vargsai, const std::vector< [Studentas](#) > &kieti)
- void [atliktiPirmaTyrima](#) ()
- void [atliktiAntraTyrima](#) ()
- template<typename Konteineris >
void [nuskaitytiFaila](#) (Konteineris &studentai, const std::string &failoVardas)
- template<typename Konteineris >
void [rikiuotiStudentus](#) (Konteineris &studentai, int pasirinkimas)

- `template<typename Konteineris >`
`void paskirstytiStudentus` (`const Konteineris &studentai`, `Konteineris &vargsai`, `Konteineris &kieti`)
- `template<typename Konteineris >`
`void paskirstytiStudentusS2` (`Konteineris &studentai`, `Konteineris &vargsai`)
- `template<typename Konteineris >`
`void paskirstytiStudentusS3` (`Konteineris &studentai`, `Konteineris &vargsai`)
- `template<typename Konteineris >`
`void atliktiAntraTyrima` (`int strategija`)

5.1.1 Funkcijos Dokumentacija

5.1.1.1 atliktiAntraTyrima() [1/2]

```
void atliktiAntraTyrima ( )
```

5.1.1.2 atliktiAntraTyrima() [2/2]

```
template<typename Konteineris >  
void atliktiAntraTyrima (  
    int strategija )
```

5.1.1.3 atliktiPirmaTyrima()

```
void atliktiPirmaTyrima ( )
```

5.1.1.4 generuotiFaila()

```
void generuotiFaila (  
    int studentuSk )
```

5.1.1.5 generuotiPazymius()

```
void generuotiPazymius (  
    Studentas & s )
```

5.1.1.6 generuotiStudentus()

```
void generuotiStudentus (  
    std::vector< Studentas > & studentai )
```

5.1.1.7 isvestiDuFailus()

```
void isvestiDuFailus (  
    const std::vector< Studentas > & vargsai,  
    const std::vector< Studentas > & kietai )
```

5.1.1.8 nuskaitytiFaila()

```
template<typename Konteineris >
void nuskaitytiFaila (
    Konteineris & studentai,
    const std::string & failoVardas )
```

5.1.1.9 nuskaitytiFailaTestavimui()

```
void nuskaitytiFailaTestavimui (
    std::vector< Studentas > & studentai,
    int kartai )
```

5.1.1.10 paskirstytiStudentus()

```
template<typename Konteineris >
void paskirstytiStudentus (
    const Konteineris & studentai,
    Konteineris & vargsai,
    Konteineris & kieti )
```

5.1.1.11 paskirstytiStudentusS2()

```
template<typename Konteineris >
void paskirstytiStudentusS2 (
    Konteineris & studentai,
    Konteineris & vargsai )
```

5.1.1.12 paskirstytiStudentusS3()

```
template<typename Konteineris >
void paskirstytiStudentusS3 (
    Konteineris & studentai,
    Konteineris & vargsai )
```

5.1.1.13 rikiuotiPagalGalutiniMed()

```
bool rikiuotiPagalGalutiniMed (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.14 rikiuotiPagalGalutiniVid()

```
bool rikiuotiPagalGalutiniVid (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.15 rikiuotiPavarde()

```
bool rikiuotiPavarde (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.16 rikiuotiStudentus()

```
template<typename Konteineris >
void rikiuotiStudentus (
    Konteineris & studentai,
    int pasirinkimas )
```

5.1.1.17 rikiuotiVarda()

```
bool rikiuotiVarda (
    const Studentas & A,
    const Studentas & B )
```

5.1.1.18 rodytiRez()

```
void rodytiRez (
    const std::vector< Studentas > & studentai )
```

5.1.1.19 rodytiRezultatus()

```
void rodytiRezultatus (
    const std::vector< Studentas > & studentai )
```

5.2 my_lib.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef MYLIB_H
00002 #define MYLIB_H
00003
00004 #include "Studentas.h"
00005
00006 #include <string>
00007 #include <vector>
00008 #include <deque>
00009 #include <list>
00010 #include <fstream>
00011 #include <sstream>
00012 #include <algorithm>
00013 #include <stdexcept>
00014 #include <iostream>
00015 #include <iomanip>
00016 #include <chrono>
00017 #include <type_traits>
00018
00019 void rodytiRezultatus(const std::vector<Studentas>& studentai);
00020 void generuotiPazymius(Studentas& s);
00021 void generuotiStudentus(std::vector<Studentas>& studentai);
00022 void rodytiRez(const std::vector<Studentas>& studentai);
00023 bool rikiuotiVarda(const Studentas& A, const Studentas& B);
00024 bool rikiuotiPavarde(const Studentas& A, const Studentas& B);
```

```

00025 bool rikiuotiPagalGalutiniVid(const Studentas& A, const Studentas& B);
00026 bool rikiuotiPagalGalutiniMed(const Studentas& A, const Studentas& B);
00027 void nuskaitytiFailaTestavimui(std::vector<Studentas>& studentai, int kartai);
00028 void generuotiFaila(int studentuSk);
00029 void isvestiDuFailus(const std::vector<Studentas>& vargsai, const std::vector<Studentas>& kieti);
00030 void atliktiPirmaTyrima();
00031 void atliktiAntraTyrima();
00032
00033 // templates
00034
00035 template <typename Konteineris>
00036 void nuskaitytiFaila(Konteineris& studentai, const std::string& failoVardas)
00037 {
00038     std::ifstream failas(failoVardas);
00039     if (!failas) {
00040         throw std::runtime_error("Nepavyko atidaryti failo " + failoVardas);
00041     }
00042
00043     std::stringstream buffer;
00044     buffer << failas.rdbuf();
00045
00046     std::string eilute;
00047     std::getline(buffer, eilute);
00048
00049     while (std::getline(buffer, eilute)) {
00050         std::stringstream ss(eilute);
00051
00052         Studentas s;
00053         std::string vardas, pavarde;
00054         if (!(ss >> vardas >> pavarde)) {
00055             throw std::runtime_error("Blogas formatas eiluteje " + eilute);
00056         }
00057         s.setVardas(vardas);
00058         s.setPavarde(pavarde);
00059
00060         int pazymys;
00061         while (ss >> pazymys) {
00062             s.pridetiNd(pazymys);
00063         }
00064
00065         if (s.getNd().empty()) {
00066             throw std::runtime_error("Truksta pazymiu eiluteje " + eilute);
00067         }
00068
00069         s.setEgz(s.getNd().back());
00070         s.pasalintiPaskutiniNd();
00071
00072         s.setGalVid(s.skaiciuotiGalutiniSuVid());
00073         s.setGalMed(s.skaiciuotiGalutiniSuMed());
00074
00075         studentai.push_back(s);
00076     }
00077 }
00078
00079 template <typename Konteineris>
00080 void rikiuotiStudentus(Konteineris& studentai, int pasirinkimas)
00081 {
00082     if (pasirinkimas < 1 || pasirinkimas > 4) {
00083         throw std::out_of_range("Pasirinkimas turi buti 1-4");
00084     }
00085
00086     if (pasirinkimas == 1) {
00087         // perziuri kompiliavimo metu, kad butu kompiliuojamas tik reikalingas rikiavimas, ziuredamas
00088         i template type
00089         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00090             studentai.sort(rikiuotiVarda);
00091         else std::sort(studentai.begin(), studentai.end(), rikiuotiVarda);
00092     } else if (pasirinkimas == 2) {
00093         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00094             studentai.sort(rikiuotiPavarde);
00095         else std::sort(studentai.begin(), studentai.end(), rikiuotiPavarde);
00096     } else if (pasirinkimas == 3) {
00097         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00098             studentai.sort(rikiuotiPagalGalutiniVid);
00099         else std::sort(studentai.begin(), studentai.end(), rikiuotiPagalGalutiniVid);
00100     } else {
00101         if constexpr (std::is_same<Konteineris, std::list<Studentas> >::value)
00102             studentai.sort(rikiuotiPagalGalutiniMed);
00103         else std::sort(studentai.begin(), studentai.end(), rikiuotiPagalGalutiniMed);
00104     }
00105 }
00106
00107 // strategija 1
00108 template <typename Konteineris>
00109 void paskirstytiStudentus(const Konteineris& studentai, Konteineris& vargsai,
00110                             Konteineris& kieti) {

```

```

00107     for (const auto& s : studentai) {
00108         if (s.getGalVid() >= 5.0) {
00109             kieti.push_back(s);
00110         } else {
00111             vargsai.push_back(s);
00112         }
00113     }
00114 }
00115
00116 // strategija 2
00117 template <typename Konteineris>
00118 void paskirstytiStudentusS2(Konteineris& studentai, Konteineris& vargsai)
00119 {
00120     vargsai.clear();
00121
00122     // rikiuoti mazejimo tvarka kad vargsai butu gale
00123     if constexpr (std::is_same_v<Konteineris, std::list<Studentas>)>
00124     {
00125         studentai.sort([](const Studentas& a, const Studentas& b) {
00126             return a.getGalVid() > b.getGalVid();
00127         });
00128     }
00129     else
00130     {
00131         std::sort(studentai.begin(), studentai.end(),
00132             [](const Studentas& a, const Studentas& b) {
00133                 return a.getGalVid() > b.getGalVid();
00134             });
00135     }
00136
00137     while (!studentai.empty() && studentai.back().getGalVid() < 5.0)
00138     {
00139         vargsai.push_back(studentai.back());
00140         studentai.pop_back();
00141     }
00142 }
00143
00144 // strategija 3
00145 template <typename Konteineris>
00146 void paskirstytiStudentusS3(Konteineris& studentai, Konteineris& vargsai)
00147 {
00148     vargsai.clear();
00149
00150     // std::partition pertvarko konteineri taip kad visi elementai kurie tenkina salyga butu
00151     // pradzioje, o kiti gale.
00152     // Grazina iterator i pirmo elemento po true grupes pradzia
00153     auto mid = std::partition(studentai.begin(), studentai.end(),
00154         [](const Studentas& s) { return s.getGalVid() < 5.0; });
00155
00156     for (auto it = studentai.begin(); it != mid; ++it) {
00157         vargsai.push_back(*it);
00158     }
00159
00160     studentai.erase(studentai.begin(), mid);
00161 }
00162
00163 template <typename Konteineris>
00164 void atliktiAntraTyrima(int strategija) {
00165     using namespace std::chrono;
00166     std::vector<int> kiekiai = {1000, 10000, 100000, 1000000, 10000000};
00167     int rikiavimas = 3;
00168
00169     std::cout << "-----\n";
00170     std::cout << std::left << std::setw(10) << "Irasai"
00171         << std::setw(12) << "Skaitymas"
00172         << std::setw(12) << "Rikiavimas"
00173         << std::setw(12) << "Skirstymas"
00174         << "Viso\n";
00175     std::cout << "-----\n";
00176
00177     for (int n : kiekiai) {
00178         Konteineris studentai;
00179         Konteineris vargsai;
00180         Konteineris kieti;
00181         std::string failoVardas = "generuotiStud" + std::to_string(n) + ".txt";
00182
00183         auto visoPradzia = high_resolution_clock::now();
00184
00185         // 1 skaitymas
00186         auto s1 = high_resolution_clock::now();
00187         try {
00188             nuskaitytiFaila(studentai, failoVardas);
00189         } catch (const std::exception& e) {
00190             std::cerr << "Klaida: Nepavyko rasti " << failoVardas << "\n";
00191         }
00192     }

```

```

00193         continue;
00194     }
00195     auto e1 = high_resolution_clock::now();
00196
00197     // 2 rikiavimas didejimo tvarka
00198     auto s2 = high_resolution_clock::now();
00199     rikiuotiStudentus(studentai, rikiavimas);
00200     auto e2 = high_resolution_clock::now();
00201
00202     // 3 skirstymas
00203     auto s3 = high_resolution_clock::now();
00204     if (strategija==1) paskirstytiStudentus(studentai, vargsai, kieti);
00205     else if (strategija==2) paskirstytiStudentusS2(studentai, vargsai);
00206     else if (strategija==3) paskirstytiStudentusS3(studentai, vargsai);
00207     auto e3 = high_resolution_clock::now();
00208
00209     auto visoPabaiga = high_resolution_clock::now();
00210
00211     double trukme1 = duration<double>(e1 - s1).count();
00212     double trukme2 = duration<double>(e2 - s2).count();
00213     double trukme3 = duration<double>(e3 - s3).count();
00214     double trukmeViso = duration<double>(visoPabaiga - visoPradzia).count();
00215
00216     std::cout << std::left << std::setw(10) << n
00217               << std::fixed << std::setprecision(4)
00218               << std::setw(12) << trukme1
00219               << std::setw(12) << trukme2
00220               << std::setw(12) << trukme3
00221               << trukmeViso << " s\n";
00222
00223     studentai.clear();
00224     vargsai.clear();
00225     kieti.clear();
00226 }
00227 std::cout << "-----\n";
00228 }
00229
00230
00231 #endif

```

5.3 include/Studentas.h Failo Nuoroda

```

#include "Zmogus.h"
#include <stdexcept>
#include <string>
#include <vector>
#include <iostream>
#include <sstream>

```

Įtraukimo priklausomybių diagrama Studentas.h:

5.4 Studentas.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "Zmogus.h"
00005
00006 #include <stdexcept>
00007 #include <string>
00008 #include <vector>
00009 #include <iostream>
00010 #include <sstream>
00011
00012 class Studentas : public Zmogus{
00013 private:
00014     int egz_;
00015     std::vector<int> nd_;
00016     double galVid_;
00017     double galMed_;
00018
00019 public:

```

```

00020     Studentas() : Zmogus("", ""), egz_(0), nd_ {}, galVid_(0.0), galMed_(0.0) {}
00021     Studentas(const std::string& vardas, const std::string& pavarde, int egz, const std::vector<int>&
nd)
00022         : Zmogus(vardas, pavarde), egz_(egz), nd_{nd}, galVid_(0.0), galMed_(0.0) {}
00023
00024     // copy konstruktorius
00025     Studentas(const Studentas& s)
00026         : Zmogus(s.vardas_, s.pavarde_),
00027         egz_(s.egz_), nd_(s.nd_),
00028         galVid_(s.galVid_), galMed_(s.galMed_) {}
00029
00030     // move konstruktorius
00031     Studentas(Studentas&& s) // && - rvalue nuoroda, std::move pavercia i rvalue
00032         : Zmogus(std::move(s.vardas_), std::move(s.pavarde_),
00033         egz_(s.egz_), nd_(std::move(s.nd_)),
00034         galVid_(s.galVid_), galMed_(s.galMed_) {
00035         s.egz_ = 0;
00036         s.galVid_ = 0.0;
00037         s.galMed_ = 0.0;
00038     }
00039
00040     // destruktorius
00041     ~Studentas() override {
00042         vardas_.clear();
00043         pavarde_.clear();
00044         nd_.clear();
00045         egz_ = 0;
00046         galVid_ = 0.0;
00047         galMed_ = 0.0;
00048     }
00049
00050     // copy assignment operatorius
00051     Studentas& operator=(const Studentas& s) {
00052         if (this == &s) return *this;
00053         vardas_ = s.vardas_;
00054         pavarde_ = s.pavarde_;
00055         egz_ = s.egz_;
00056         nd_ = s.nd_;
00057         galVid_ = s.galVid_;
00058         galMed_ = s.galMed_;
00059
00060         return *this;
00061     }
00062
00063     // move assignment operatorius
00064     Studentas& operator=(Studentas&& s) {
00065         if (this == &s) return *this;
00066         vardas_ = std::move(s.vardas_);
00067         pavarde_ = std::move(s.pavarde_);
00068         nd_ = std::move(s.nd_);
00069         egz_ = s.egz_;
00070         galVid_ = s.galVid_;
00071         galMed_ = s.galMed_;
00072
00073         s.egz_ = 0;
00074         s.galVid_ = 0.0;
00075         s.galMed_ = 0.0;
00076
00077         return *this;
00078     }
00079
00080     std::string tipas() const override { return "Studentas"; }
00081
00082     // output operatorius
00083     friend std::ostream& operator<<(std::ostream& out, const Studentas& s) {
00084         out << s.vardas_ << ' ' << s.pavarde_;
00085         for (int nd : s.nd_) out << ' ' << nd;
00086         out << ' ' << s.egz_;
00087         return out;
00088     }
00089
00090     // input operatorius
00091     friend std::istream& operator>>(std::istream& in, Studentas& s) {
00092         std::string eilute;
00093         std::getline(in, eilute);
00094         if (!in) return in;
00095
00096         std::stringstream ss(eilute);
00097
00098         if (!(ss >> s.vardas_ >> s.pavarde_)) {
00099             throw std::runtime_error("Blogas studento formatas: truksta vardo/pavardes");
00100         }
00101
00102         s.nd_.clear();
00103         int paz;
00104         while (ss >> paz) s.nd_.push_back(paz);
00105

```

```

00106     if (s.nd_.empty()) {
00107         throw std::runtime_error("Blogas studento formatas: truksta pazymiu");
00108     }
00109
00110     s.egz_ = s.nd_.back();
00111     s.nd_.pop_back();
00112
00113     s.galVid_ = s.skaiciuotiGalutiniSuVid();
00114     s.galMed_ = s.skaiciuotiGalutiniSuMed();
00115     return in;
00116 }
00117
00118 int getEgz() const { return egz_; }
00119 const std::vector<int>& getNd() const { return nd_; }
00120 double getGalVid() const { return galVid_; }
00121 double getGalMed() const { return galMed_; }
00122
00123 void setEgz(int e) { egz_ = e; }
00124 void setGalVid(double v) { galVid_ = v; }
00125 void setGalMed(double v) { galMed_ = v; }
00126
00127 void pridetiNd(int pazymys) { nd_.push_back(pazymys); }
00128 void isvalytiNd() { nd_.clear(); }
00129 void pasalintiPaskutiniNd() {
00130     if (nd_.empty()) throw std::runtime_error("Truksta pazymiu eiluteje");
00131     nd_.pop_back();
00132 }
00133
00134
00135 double skaiciuotiGalutiniSuVid() const;
00136 double skaiciuotiGalutiniSuMed() const;
00137 };
00138
00139 #endif
00140

```

5.5 include/Zmogus.h Failo Nuoroda

#include <string>

Įtraukimo priklausomybių diagrama Zmogus.h: Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:

Klasės

- class [Zmogus](#)

5.6 Zmogus.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00006 class Zmogus {
00007 protected:
00008     std::string vardas_;
00009     std::string pavarde_;
00010
00011 public:
00012     Zmogus() : vardas_(""), pavarde_("") {}
00013
00014     Zmogus(std::string vardas, std::string pavarde) : vardas_(vardas), pavarde_(pavarde) {};
00015
00016     virtual ~Zmogus() {
00017         vardas_.clear();
00018         pavarde_.clear();
00019     };
00020
00021     // pure virtual funkcija pavercia klase abstrakcia
00022     virtual std::string tipas() const = 0;
00023

```

```

00024     const std::string& getVardas() const { return vardas_; }
00025     const std::string& getPavarde() const { return pavarde_; }
00026
00027     void setVardas(const std::string& v) { vardas_ = v; }
00028     void setPavarde(const std::string& p) { pavarde_ = p; }
00029 };
00030 #endif

```

5.7 src/main.cpp Failo Nuoroda

```

#include "my_lib.h"
#include <cctype>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
#include <ctime>

```

Įtraukimo priklausomybių diagrama main.cpp:

Funkcijos

- int [main](#) ()

5.7.1 Funkcijos Dokumentacija

5.7.1.1 main()

```
int main ( )
```

5.8 src/my_lib.cpp Failo Nuoroda

```

#include "my_lib.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <vector>
#include <cctype>
#include <chrono>
#include <sstream>
#include <fstream>

```

Įtraukimo priklausomybių diagrama my_lib.cpp:

Funkcijos

- void [rodytiRezultatus](#) (const std::vector< [Studentas](#) > &studentai)
- void [generuotiPazymius](#) ([Studentas](#) &s)
- void [generuotiStudentus](#) (std::vector< [Studentas](#) > &studentai)
- void [rodytiRez](#) (const std::vector< [Studentas](#) > &studentai)
- bool [rikiuotiVarda](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- bool [rikiuotiPavarde](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- bool [rikiuotiPagalGalutiniVid](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- bool [rikiuotiPagalGalutiniMed](#) (const [Studentas](#) &A, const [Studentas](#) &B)
- void [nuskaitytiFailaTestavimui](#) (std::vector< [Studentas](#) > &studentai, int kartai)
- void [generuotiFaila](#) (int studentuSk)
- void [isvestiDuFailus](#) (const std::vector< [Studentas](#) > &vargsai, const std::vector< [Studentas](#) > &kietai)
- void [atliktiPirmaTyrima](#) ()

5.8.1 Funkcijos Dokumentacija

5.8.1.1 atliktiPirmaTyrima()

```
void atliktiPirmaTyrima ( )
```

5.8.1.2 generuotiFaila()

```
void generuotiFaila (
    int studentuSk )
```

5.8.1.3 generuotiPazymius()

```
void generuotiPazymius (
    Studentas & s )
```

5.8.1.4 generuotiStudentus()

```
void generuotiStudentus (
    std::vector< Studentas > & studentai )
```

5.8.1.5 isvestiDuFailus()

```
void isvestiDuFailus (
    const std::vector< Studentas > & vargsai,
    const std::vector< Studentas > & kieti )
```

5.8.1.6 nuskaitytiFailaTestavimui()

```
void nuskaitytiFailaTestavimui (
    std::vector< Studentas > & studentai,
    int kartai )
```

5.8.1.7 rikiuotiPagalGalutiniMed()

```
bool rikiuotiPagalGalutiniMed (
    const Studentas & A,
    const Studentas & B )
```

5.8.1.8 rikiuotiPagalGalutiniVid()

```
bool rikiuotiPagalGalutiniVid (
    const Studentas & A,
    const Studentas & B )
```

5.8.1.9 rikiuotiPavarde()

```
bool rikiuotiPavarde (
    const Studentas & A,
    const Studentas & B )
```

5.8.1.10 rikiuotiVarda()

```
bool rikiuotiVarda (
    const Studentas & A,
    const Studentas & B )
```

5.8.1.11 rodytiRez()

```
void rodytiRez (
    const std::vector< Studentas > & studentai )
```

5.8.1.12 rodytiRezultatus()

```
void rodytiRezultatus (
    const std::vector< Studentas > & studentai )
```